

RELAZIONE HACKHUB

Hackhub è una piattaforma web creata per supportare la gestione e la partecipazione ad hackathon a squadre. Sono presenti cinque tipologie di utenti: organizzatore, utente registrato, utente non registrato, giudice e mentore. Le principali funzionalità del sistema comprendono la creazione di hackathon e team, l'iscrizione dei team agli eventi, l'invio delle sottomissioni, la valutazione dei progetti, il supporto da parte dei mentori e la determinazione dei vincitori. Non tutte le funzionalità sono presenti nel frontend, in quanto il backend è stato sviluppato precedentemente per un progetto molto più ampio. A livello di frontend è possibile creare un hackathon, visualizzare gli hackathon sia nelle card presenti nella dashboard con nome, data di inizio e di fine e luogo, sia in una pagina dedicata alla visualizzazione di hackathon con tutte le informazioni legate a quell'evento. È possibile inoltre visualizzare i team presenti all'interno della piattaforma, oltre alla possibilità di registrarsi e autenticarsi. Il frontend mostrato è la pagina tipica di un organizzatore.

L'applicazione è composta da tre livelli principali:

- frontend ovvero l'interfaccia utente che interagisce con il backend, sviluppata come Single Page Application;
- backend, che implementa la logica di business, gestisce la sicurezza dell'applicazione ed espone le API REST
- Database, utilizzato per la persistenza dei dati relativi agli utenti e a tutte le altre entità del sistema.

FRONTEND

Le tecnologie usate per sviluppare il frontend sono:

- React, una libreria Javascript usata per lo sviluppo dell'interfaccia. L'interfaccia è organizzata in pagine, componenti riutilizzabili e hook personalizzati per la gestione dello stato e della navigazione. Le pagine sviluppate sono la dashboard, la visualizzazione di team, di hackathon e la pagina di login/registrazione. È inoltre presente una sidebar riutilizzabile che consente la navigazione tra le diverse sezioni dell'applicazione.
- Vite, strumento utilizzato per la build. È presente all'interno del frontend un file vite.config.js che utilizza la funzionalità di Proxy offerta da Vite. Le richieste indirizzate al percorso /api vengono automaticamente inoltrate al backend sulla porta 8081 in modo da separare frontend e backend garantendo una comunicazione sicura e semplificata tra client e server.
- Nginx, utilizzato per servire i file statici generati dalla build React in modo da migliorare le prestazioni nella gestione di richieste HTTP. Le richieste indirizzate al percorso /api vengono inoltrate come vite al backend, consentendo al frontend e al

backend di essere esposti tramite un unico punto di accesso riducendo la necessità di configurazioni CORS.

I file presenti in Vite e Nginx gestiscono lo stesso problema ma lo fanno in due ambienti differenti. Il proxy di vite viene utilizzato solo durante lo sviluppo con un localhost mentre nginx viene usato in produzione tramite Docker.

BACKEND

Il backend è sviluppato seguendo un'architettura di tipo MVC. Le principali componenti sono:

- boundary: espongono le API REST e rappresentano il punto di accesso alle funzionalità di tutta l'applicazione
- service: implementano la logica di business e gestiscono le operazioni per soddisfare le richieste degli utenti
- repository: gestiscono l'accesso ai dati e al database
- entity: rappresentano il modello persistente usato da JPA/Hibernate per la generazione dello schema del database

Le tecnologie principali usate per sviluppare il backend sono:

- Java 21: linguaggio principale scelto per la facile integrazione con Spring
- Spring Boot: costituisce il framework principale, usato per l'esposizione delle API REST, l'utilizzo di metodi predefiniti per le repository e la gestione generale dell'applicazione
- Spring Security: implementato per la gestione dell'autenticazione e autorizzazione degli utenti. Mi permette di creare dei vincoli di accesso alle risorse e di integrare se necessario altri meccanismi di sicurezza
- JWT: sistema di autenticazione stateless basato su token (in particolare in questa applicazione viene utilizzato un Bearer Token). Dopo il login il server genera il token con le informazioni dell'utente e tramite questo l'utente accede alla piattaforma. Necessario per l'utilizzo di tutte le API REST tranne la visualizzazione di info pubbliche (in quanto le info pubbliche sono visualizzabili anche da utenti non registrati)
- Spring Data JPA: per la gestione della persistenza dei dati, semplificando l'interazione con il database e riducendo la quantità di codice necessario per operazioni CRUD. Usato insieme ad Hibernate per gestire automaticamente il mapping tra entità Java e tabelle relazionali

DATABASE

Per la creazione del database è stato utilizzato MySQL, soprattutto per la persistenza dei dati. Tutte le informazioni relative alle entity presenti nel modello di dominio sono memorizzate all'interno del database e le relazioni sono gestite automaticamente tramite JPA e Hibernate.

Durante lo sviluppo il popolamento iniziale del database è stato automatizzato tramite il file `data.sql`, che inserisce dati di esempio relativi a utenti, team e hackathon. Ciò consente di testare rapidamente le funzionalità dell'applicazione senza dover inserire manualmente i dati ad ogni avvio.

CONTAINERIZZAZIONE, TEST E DEPLOY

Per semplificare l'esecuzione in ambienti differenti è stato utilizzato Docker. Sono stati creati tre container separati, frontend, backend e database, per isolare le dipendenze e distribuire velocemente l'applicazione su altri sistemi operativi.

Viene inoltre utilizzato GitHub Actions per implementare una pipeline di CI che esegue test ad ogni commit.